

FIAP

Global Solution – Data Structures and Algorithms: Sistema de
Monitoramento e Análise de Dados de uma Missão Espacial

São Paulo – 2026

1CCPG - 2026

Enzo Caruso Peter – RM: 570908

Leonardo Robert Maulicino – RM: 570329

Lucas Ramos de Sousa – RM: 573901

São Paulo – 2026

Sumário

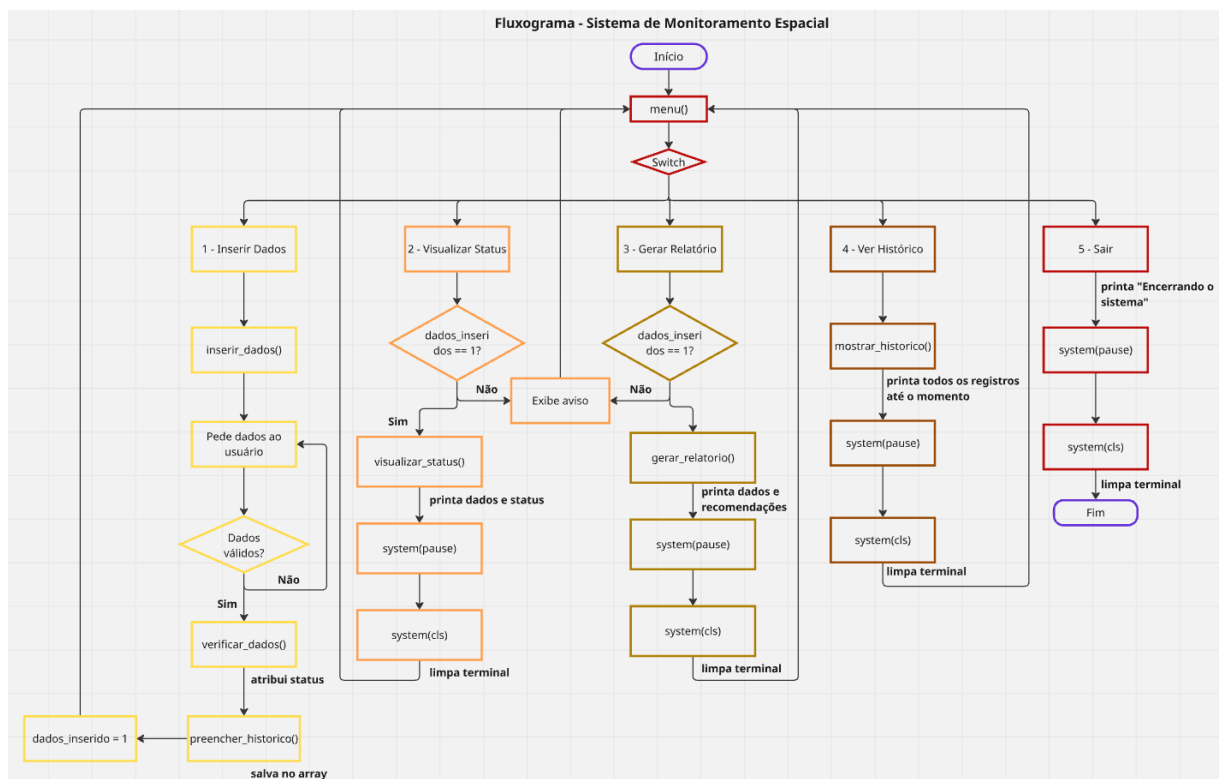
1.	Contextualização	4
2.	Fluxograma	4
3.	Lógica em C	5
3.1.	Bibliotecas, Constantes e Variáveis Globais	5
3.2.	Função verificar_dados()	6
3.3.	Função inserir_dados()	7
3.4.	Função visualizar_status()	8
3.5.	Função gerar_relatorio()	9
3.6.	Função preencher_historico()	10
3.7.	Função mostrar_historico()	10
3.8.	Função menu()	12
3.9.	Função main()	13
4.	Estruturas Utilizadas:	14

1. Contextualização

Esse projeto consiste em um sistema em C capaz de monitorar informações básicas de uma missão espacial experimental, utilizando estruturas condicionais, repetição, vetores e funções para análise de dados operacionais simulados.

Em missões espaciais, sistemas computacionais monitoram constantemente informações importantes para garantir o funcionamento correto da operação. Esse programa: recebe dados simulados (temperatura, energia e comunicação); analisa condições da missão; emite alertas básicos; organiza informações no terminal

2. Fluxograma



*Obs: imagem muito pequena, por favor, acesso arquivo via link ou imagem enviada junto do projeto.

Link para acesso melhor ao fluxograma:

https://miro.com/app/board/uXjVHY5oUnc=?share_link_id=263078877482

3. Lógica em C

3.1. Bibliotecas, Constantes e Variáveis Globais

```
// Adicionando as bibliotecas necessárias
#include<stdio.h>
#include <stdlib.h> //Função system() funciona apenas no Windows

// Cores de texto
#define VERMELHO "\033[31m"
#define VERDE "\033[32m"
#define AMARELO "\033[33m"
#define RESET "\033[0m"

//Definindo constantes
#define TEMP_MAX 80.0
#define ENERGIA_MIN 20.0
#define MAX_HISTORICO 5

//Criando struct
typedef struct {
    float temperatura;
    float energia;
    int comunicacao;
    int status_temp; //0 - > temperatura máx. | 1 - normal
    int status_ene; //0 - < energia mín. | 1 - normal
} Dados_nave;

Dados_nave historico[MAX_HISTORICO];

int total_registros = 0;
int proximo_indice = 0;
```

- Bibliotecas:
 - stdio: para funções de printf() e scanf() de interação com o usuário;
 - stdlib: para função system() de comandos no sistema;
- Cores: para personalizar o terminal;
- Constantes:
 - TEMP_MAX: define a temperatura máxima que o sistema suporta antes de dar alerta;
 - ENERGIA_MIN: define a % de energia mínima que o sistema suporta antes de dar alerta;
 - MAX_HISTORICO: define a quantidade máxima de registros que o histórico suporta
- Struct Dados_nave: objeto para organizar todas as variáveis que cada registro do sistema pode ter;
- Int total_registros: variável para guardar quantos registros já foram preenchidos;
- Int proximo_indice: variável para guardar qual o próximo índice vazio;

3.2. Função verificar_dados()

```
//Função para verificar estado das variáveis
void verificar_dados(Dados_nave *nave){
    if(nave->temperatura > TEMP_MAX){
        nave->status_temp = 0;
    } else{
        nave->status_temp = 1;
    }

    if(nave->energia < ENERGIA_MIN){
        nave->status_ene = 0;
    } else{
        nave->status_ene = 1;
    }
}
```

- Recebe a Struct que vai trabalhar;
- Verifica se a temperatura da nave ultrapassou o limite definido e altera o status para 0 (crítico) ou 1 (normal);
- Verifica se a energia da nave está abaixo do limite definido e altera o status para 0 (crítico) ou 1 (normal);

3.3. Função inserir_dados()

```
//Função para pegar os dados simulados
void inserir_dados(Dados_nave *nave){
    printf(AMARELO "-----\n" RESET);
    printf(AMARELO "    Inserindo Dados    \n" RESET);
    printf(AMARELO "-----\n" RESET);
    printf("\n");

    do{
        printf("Digite a temperatura interna da nave: ");
        scanf("%f", &nave->temperatura);
        printf("\n");
    } while(nave->temperatura < 0.0 || nave->temperatura > 100.0);

    do{
        printf("Digite a %% de energia da nave: ");
        scanf("%f", &nave->energia);
        printf("\n");
    } while(nave->energia < 0.0 || nave->energia > 100.00);

    do{
        printf("Digite o status de comunicacao da nave: \n");
        printf("0 - Falha\n");
        printf("1 - Ativa\n");
        scanf("%d", &nave->comunicacao);
        printf("\n");
    } while(nave->comunicacao < 0 || nave->comunicacao > 1);

    verificar_dados(nave);
}
```

- Recebe a Struct que vai trabalhar;
- Pede ao usuário a temperatura atual da nave, a % de energia atual e o status de comunicação com a central;
- Para cada input verifica se foi digitado um valor válido;
- Chama a função verificar dados para preencher as variáveis de status;

3.4. Função visualizar_status()

```
//Função para visualizar as variáveis e o estado de cada uma
void visualizar_status(Dados_nave *nave){
    printf(AMARELO "-----\n" RESET);
    printf(AMARELO "    Visualizar Dados    \n" RESET);
    printf(AMARELO "-----\n" RESET);
    printf("\n");

    printf("Temperatura Interna: %.1f C | Status: ", nave->temperatura);
    if(nave->status_temp == 0){
        printf( VERMELHO "Superaquecimento\n" RESET);
    }else{
        printf( VERDE "Normal\n" RESET);
    }
    printf("\n");

    printf("Energia da Nave: %.1f%% | Status: ", nave->energia);
    if(nave->status_ene == 0){
        printf( VERMELHO "Economia de Energia\n" RESET);
    }else{
        printf( VERDE "Normal\n" RESET);
    }
    printf("\n");

    printf("Comunicacao com a central: ");
    if(nave->comunicacao == 0){
        printf( VERMELHO "Falhou\n" RESET);
    }else{
        printf( VERDE "Ativa\n" RESET);
    }
    printf("\n");
}
```

- Recebe a Struct que vai trabalhar;
- Printa no terminal os últimos valores inseridos pelo usuário e o status de cada dado com base na verificação feita;

3.5. Função gerar_relatorio()

```
//Função para gerar um relatório com as ações necessárias para cada variável
void gerar_relatorio(Dados_nave *nave){
    printf(AMARELO "-----\n" RESET);
    printf(AMARELO "          Relatorio          \n" RESET);
    printf(AMARELO "-----\n" RESET);
    printf("\n");

    printf("Temperatura Interna: %.1f C\n", nave->temperatura);
    printf("Acoes: ");
    if(nave->status_temp == 0){
        printf( VERMELHO "Desligue o motor e procure a causa do aumento de
        temperatura.\n" RESET);
    }else{
        printf( VERDE "Sem acoes necessarias.\n" RESET);
    }
    printf("\n");

    printf("Energia da Nave: %.1f%%\n", nave->energia);
    printf("Acoes: ");
    if(nave->status_ene == 0){
        printf( VERMELHO "Pare a nave, desligue equipamentos menos
        prioritarios, espere o carregamento por energia solar.\n" RESET);
    }else{
        printf( VERDE "Sem acoes necessarias.\n" RESET);
    }
    printf("\n");

    printf("Comunicacao com a central: ");
    if(nave->comunicacao == 0){
        printf( VERMELHO "Falhou\n" RESET);
    }else{
        printf( VERDE "Ativa\n" RESET);
    }
    printf("Acoes: ");
    if(nave->comunicacao == 0){
        printf( VERMELHO "Reestabeleca comunicacao com a central urgentemente.
        \n" RESET);
    }else{
        printf( VERDE "Sem acoes necessarias.\n" RESET);
    }
    printf("\n");
}
```

- Recebe a Struct que vai trabalhar;
- Printa no terminal os últimos valores inseridos pelo usuário e a ação recomendada a se fazer com base no status de cada dado;

3.6. Função preencher_historico()

```
//Função para preencher o histórico de dados da nave, mantendo os últimos 5
registros (Antigo -> Recente)
void preencher_historico(Dados_nave *nave){
    historico[proximo_indice] = *nave;

    proximo_indice = (proximo_indice + 1) % MAX_HISTORICO;

    if(total_registros < MAX_HISTORICO){
        total_registros++;
    }
}
```

- Recebe a Struct que vai trabalhar;
- Preenche a primeira posição livre do histórico com a Struct atual;
- Altera o próximo índice sempre limitando com base no máximo de históricos;
- Aumenta o total de registros preenchidos enquanto ele for menor que o máximo;

3.7. Função mostrar_historico()

```
//Função para mostrar o histórico de dados da nave (Antigo -> Recente)
void mostrar_historico(){
    printf(AMARELO "-----\n" RESET);
    printf(AMARELO "      Historico      \n" RESET);
    printf(AMARELO "-----\n" RESET);
    printf("\n");

    if(total_registros == 0){
        printf("Nenhum registro no historico.\n\n");
        return;
    }

    int indice_inicial;
    if(total_registros < MAX_HISTORICO){
        indice_inicial = 0;
    } else {
        indice_inicial = proximo_indice;
    }
}
```

- Printa a mensagem de nenhum registro caso nenhum tenha sido preenchido ainda;
- Verifica se o limite de registros foi atingido e zera a contagem para substituir registros mais antigos;

```

for(int i = 0; i < total_registros; i++){
    int indice = (indice_inicial + i) % MAX_HISTORICO;

    printf("Registro %d:\n", i + 1);
    printf("Temperatura Interna: %.1f C | Status: ", historico[indice].
    temperatura);
    if(historico[indice].status_temp == 0){
        printf( VERMELHO "Superaquecimento\n" RESET);
    }else{
        printf( VERDE "Normal\n" RESET);
    }
    printf("\n");

    printf("Energia da Nave: %.1f%% | Status: ", historico[indice].energia);
    if(historico[indice].status_ene == 0){
        printf( VERMELHO "Economia de Energia\n" RESET);
    }else{
        printf( VERDE "Normal\n" RESET);
    }
    printf("\n");

    printf("Comunicacao com a central: ");
    if(historico[indice].comunicacao == 0){
        printf( VERMELHO "Falhou\n" RESET);
    }else{
        printf( VERDE "Ativa\n" RESET);
    }
    printf("\n");
    printf("-----\n\n");
}

if(total_registros == MAX_HISTORICO){
    printf("Historico cheio - os registros mais antigos estao sendo
    substituidos\n\n");
}
}

```

- Para cada registro armazenado, printa seu número, seus dados (temperatura, energia e comunicação) e o status de cada um;
- Caso chegue no limite de histórico ele avisa que os dados antigos serão substituídos;

3.8. Função menu()

```
//Função menu
int menu(){
    int opcao;

    do{
        printf(AMARELO "=====\n" RESET);
        printf(AMARELO "  Missao Espacial  \n" RESET);
        printf(AMARELO "=====\n" RESET);
        printf("\n");

        printf("1 - Inserir Dados\n");
        printf("2 - Visualizar Status\n");
        printf("3 - Gerar Relatorio\n");
        printf("4 - Ver Historico\n");
        printf("5 - Sair\n");
        printf("Escolha uma opcao: ");
        scanf("%d", &opcao);
        printf("\n");

    } while(opcao < 1 || opcao > 5);

    return opcao;
}
```

- Printa um menu com as opções de acesso ao usuário e retorna a opção escolhida;
- Valida se o usuário digitou uma opção válida;

3.9. Função main()

```
int main(){
    int opcao;
    int dados_inseridos = 0;
    Dados_nave minha_nave;

    do{
        switch(opcao = menu()){
            case 1:
                inserir_dados(&minha_nave);

                preencher_historico(&minha_nave);

                dados_inseridos = 1;
                break;

            case 2:
                if(dados_inseridos == 1){
                    visualizar_status(&minha_nave);

                    system("pause");
                    system("cls");
                } else {
                    printf("Insira os dados primeiro (opcao 1).\n\n");
                }
                break;
        }
    }
```

- Cria uma cópia da Struct para ser atualizada;
- Chama o menu para saber qual opção o usuário deseja;
- Opção 1:
 - Chama a função `inserir_dados()`;
 - Chama a função `preencher_historico()`;
 - Marca que tem dados inseridos no sistema;
- Opção 2:
 - Caso houver dados preenchidos no sistema, chama `visualizar_status()`;
 - Pausa o terminal e depois limpa o sistema;

```

        case 3:
            if(dados_inseridos == 1){
                gerar_relatorio(&minha_nave);

                system("pause");
                system("cls");
            } else {
                printf("Insira os dados primeiro (opcao 1).\n\n");
            }
            break;

        case 4:
            mostrar_historico();

            system("pause");
            system("cls");
            break;
    }
} while(opcao != 5);

printf("Encerrando o sistema...");
printf("\n");

system("pause");
system("cls");

return 0;
}

```

- Opção 3:
 - Caso houver dados preenchidos no sistema, chama gerar_relatorio();
 - Pausa o terminal e depois limpa o sistema;
- Opção 4:
 - Chama mostrar_historico();
 - Pausa o terminal e depois limpa o sistema;
- Opção 5:
 - Sai do loop, pausa e limpa o terminal e encerra o sistema;

4. Estruturas Utilizadas:

- For: em mostrar_historico() para passar por todos os dados registrados no array historico;
- If/Else: em verificar_dados(), visualizar_status(), gerar_relatorio(), preencher_historico() e mostrar_historico() para verificar o status de cada dado e printar a mensagem respectiva;
- Do/While: em inserir_dados, menu() e main() para validações das entradas do usuário e para manter o sistema ativo enquanto o usuário não sair;
- System(): funções system() funcionam apenas no Windows;